

Yazılım Yaşam Döngüsü Gerçekleştirim

Yazılım Mühendisliği Dersi

Giriş

- Tasarım sonucu üretilen süreç ve veritabanının fiziksel yapısını içeren fiziksel modelin bilgisayar ortamında çalışan yazılım biçimine dönüştürülmesi çalışmasıdır.
- Herşeyden önce bir yazılım geliştirme ortamı seçilmelidir (programlama dili, veritabanı yönetim sistemi, yazılım geliştirme araçları (CASE)).
- Kaynak kodların belirli bir standartta üretilmesi düzeltme için faydalıdır.

Programlama Dilleri

- Geliştirilecek uygulamaya göre programlama dilleri seçilmelidir.
 - **Veri işleme yoğunluklu uygulamalar**
 - Cobol
 - Görsel programlama dilleri ve veritabanları
 - **Hesaplama yoğunluklu uygulamalar**
 - Fortran
 - C
 - **Süreç ağırlıklı uygulamalar**
 - Assembly
 - C
 - **Sistem programlamaya yönelik uygulamalar**
 - C
 - **Yapay zeka uygulamaları**
 - Lisp
 - Prolog

Veri Modelleri

- Fiziksel veri modelleri verinin bilgisayarda nasıl saklandığının detayları ile ilgili kavramları sağlar.
- Yüksek düzey veri modelleri
 - **Varlık:** Veritabanında saklanan, gerçek dünyadan bir nesne veya kavramdır (proje, işçi).
 - **Özellik:** Varlığı anlatan bir özelliği gösterir (işçinin adı, ücreti).
 - **İlişki:** Birden fazla varlık arasındaki ilişkidir (işçi ve proje arasındaki çalışma ilişkisi).

Şemalar

- Herhangi bir veri modelinde veritabanının tanımlaması ile kendisini ayırmak önemlidir.
- Veritabanının tanımlaması, veritabanı şeması veya meta-veri olarak adlandırılır.
- Veritabanı şeması tasarım sırasında belirlenir ve fazla değişmesi beklenmez.

VTYS Mimarisi

- VTYS mimarisi üç seviyede tanımlanabilir.
 - **İşsel düzey:** Veritabanını fiziksel saklama yapısını açıklar.
 - **Kavramsal düzey:** Kavramsal şema içerir ve kullanıcılar için veritabanının yapısını açıklar.
 - **Dışsal düzey:** Dış şemalar ve kullanıcı görüşlerini içerir.

Veritabanı Dilleri ve Arabirimleri

- Veritabanı tasarımı tamamlandıktan sonra bir VTYS seçilir.

VTYS Dilleri

Veritabanı tanımlama dili	Kavramsal şemaları tanımlama üzere kullanılır
Saklama tanımlama dili	İçsel şemayı tanımlama üzere kullanılır.
Görüş tanımlama dili	Dışsal şemayı tanımlamak için kullanılır.
Veri işleme dili	Veritabanı oluşturulduktan sonra veri eklemek, değiştirmek ve silmek için kullanılır.

VTYS nin Sınıflandırılması

- Sınıflandırmalar kullanılan veri modeline göre yapılır.
- İlişkisel model: Veritabanı tablo yığınınından oluşmuştur. Herbir tablo bir dosya olarak saklanabilir.
- Ağ modeli: Veriyi kayıt ve küme tipleri olarak gösterir.
- Hiyerarşik model: Veri ağaç yapısında gösterilir.
- Nesne yönelimli model: Veritabanı nesneler, özellikleri ve işlemleri biçiminde gösterilir.

Hazır Program Kütüphaneleri

- Hemen hemen tüm programlama platformlarının kendilerine özgü hazır kütüphaneleri bulunmaktadır.
 - Pascal *.tpu
 - C *.h
 - Java *.jar
- Günümüzde bu kütüphanelerin temin edilmesi internet üzerinden oldukça kolaydır.

CASE Araç ve Ortamları

- Günümüzde bilgisayar destekli yazılım geliştirme ortamları oldukça gelişmiştir.
- CASE araçları yazılım geliştirme sürecinin her aşamasında üretilen bilgi ya da belgelerin bilgisayar ortamında saklanmasına ve bu yolla kolay erişilebilir ve yönetilebilir olmasına olanak sağlar.

Kodlama Stili

- Hangi platformda geliştirilirse geliştirilsin yazılımın belirli bir düzende kodlanması, yazılımın yaşam döngüsü açısından önem kazanmaktadır.
- Etkin kod yazılım stili için kullanılan yöntemler:
 - Açıklama satırları
 - Kod yazım düzeni
 - Anlamlı isimlendirme
 - Yapısal programlama yapıları

Açıklama Satırları

- Modülün başlangıcına
 - Genel amaç
 - İşlevi
 - Desteklenen platformlar
 - Eksiler
 - Düzeltilen yanlışlıklar
- gibi genel bilgilendirici açıklamaları yapılır.
- Aynı zamanda modülün kritik bölümleri vurgulanarak açıklanır.

Kod Biçimlemesi

- Programın okunabilirliğini artırmak ve anlaşılabilirliğini kolaylaştırmak amacıyla açıklama satırlarının kullanımının yanısıra, belirli bir kod yazım düzeninin de kullanılması gerekmektedir.

```
for i:=1 to 50 do begin i:=i+1; end;    // kötü kodlanmış
```

```
for i:=1 to 50 do                      // iyi kodlanmış
begin
    i:=i+1;
end;
```

Anlamalı İsimlendirme

- Kullanılan tanımlayıcıların (değişken adları, dosya adları, veritabanı tablo adları, fonksiyon adları gibi) anlamlı olarak isimlendirilmeleri anlaşılabilirliği büyük ölçüde etkilemektedir.

Yapısal Programlama Yapıları

- Yapısal programlama yapıları temelde içinde `goto` komutlarının bulunmadığı, tek giriş ve tek çıkışlı öbeklerden oluşan yapılardır. Bunlar temelde;
 - Ardışıl işlem yapıları,
 - Koşullu işlem yapıları,
 - Döngü yapılarıdır.
- Teorik olarak herhangi bir bilgisayar programının sadece bu yapılar kullanılarak yazılabileceği kanıtlanmıştır.

Program Karmaşıklığı

- Program karmaşıklığı konusunda çeşitli modeller geliştirilmiştir.
- Bunların en eskisi ve yol göstericisi McCabe tarafından 1976 yılında geliştirilen modeldir.
- Bunun için önce programın akış diyagramına dönüştürülmesi, sonra da **karmaşıklık ölçütünün** hesaplanması gerekmektedir.

Akış Diyagramına Dönüştürme



Akış Diyagramına Dönüştürme (2)



McCabe Karmaşıklık Ölçütü

$$V(G) = k - d + 2p$$

K: Diyagramdaki kenar çizgi sayısı

D: Diyagramdaki düğüm sayısı

P: Programdaki bileşen sayısı (ana program ve ilgili alt program sayısını göstermektedir. Alt program kullanılmadıysa p=1, 3 alt program kullanıldıysa p=4 tür.)

Olağandışı Durum Çözümleme

- Olağandışı durum: Bir programın çalışmasının, geçersiz ya da yanlış veri oluşumu ya başka nedenlerle istenmeyen bir biçimde sonlanmasına neden olan bir durumdur.
- Genelde kabul edilen; program işletiminin sonlandırılmasının bütünüyle program denetiminde olmasıdır.

Kod Gözden Geçirme

- Bir gazete hiçbir yazı editörünün onayı alınmadan basılamayacağı gibi, kod gözden geçirme olmadan da yazılım sistemi geliştirilemez.
- Kod gözden geçirme ile program sinama işlemleri birbirlerinden farklıdır.
- Kod gözden geçirme, programın kaynak kodu üzerinde yapılan bir işlemdir ve bu işlemlerde program hatalarının %3-5 lik bir kısmı yakalanabilmektedir.